

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student & Hardware Metadata Field	Laboratory Session Specifications & Target Machine Declaration
Student Full Name	Kystaubay Ayanat
Student Identification Number	230103075
Date & Laboratory Session	17.09.2026 Advanced Parallel Programming
Workstation / Laptop Make & Model	Huawei MateBook D 14
Processor SKU / SoC Model	12th Gen Intel(R) Core(TM) i5-12450H (2.00 GHz base)
Core Topology (Physical P-cores / E-cores)	Physical Cores: 8 (4 P-cores + 4 E-cores) Logical SMT Threads: 12
Cache Hierarchy Capacity	L1: 704 KB L2: 7 MB L3: 12 MB
Host OS & Python Runtime	Windows 11 Home 25H2 (build 26200.8655). Benchmarks: Google Colab VM, Linux 6.6.122 x86-64, 2 vCPU (1 physical core) / CPython 3.13.15

MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where $S(p)$ is the speedup ratio $T(1)/T(p)$, P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

EXECUTION PROTOCOL

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline $T(1)$ and the measured multi-core execution times across all 6 core allocations.

```

import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")

```

Worker Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency [S(p)/p * 100]
p = 1 (Baseline)	2.696	1.00x	1.00x	100.0%
p = 2	2.056	1.31x	2.00x	65.6%
p = 4	1.997	1.35x	4.00x	33.7%
p = 8	1.970	1.37x	8.00x	17.1%
p = 12	1.987	1.36x	12.00x	11.3%
p = 16	2.828	0.95x	16.00x	6.0%

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

Student Response & Architectural Justification:

Empirical inflection point: $p^* = 2$.

Measurements were executed on a Google Colab VM (Linux 6.6, x86-64, 2 vCPU = 1 physical core + 1 SMT sibling, CPython 3.13.15), not on the MateBook, so all empirical analysis below refers to that VM's topology.

Observed curve: $S(2) = 1.31x$, then flat - $S(4) = 1.35x$, $S(8) = 1.37x$, $S(12) = 1.36x$ - and at $p = 16$ the program becomes slower than serial ($T = 2.828$ s, $S = 0.95x$). The knee sits exactly at the VM's 2 logical CPUs. The second logical CPU is an SMT hyperthread of the same physical core: it shares the execution engine, L1/L2 caches and memory path, so $p = 2$ delivers only 1.31x instead of 2x (65.6% efficiency). For $p > 2$ there are more processes than logical CPUs, so the OS time-slices them on the same silicon; each extra worker contributes only process spawn latency, pickle-based IPC and context switches. At $p = 16$ this overhead exceeds the compute itself and wall time rises from 1.97 s to 2.83 s.

Correlation with my laptop (Huawei MateBook D 14, Intel Core i5-12450H: 4 P-cores with Hyper-Threading + 4 E-cores = 8 physical cores / 12 logical threads): on bare metal the first knee is expected at $p = 4$ (P-cores exhausted; further workers land on slower Gracemont E-cores), strong flattening at $p = 8$ (all physical cores busy), and no gain beyond $p = 12$ (only SMT siblings/oversubscription remain).

[Q1.2] Using your empirical measurements for $T(1)$ and $T(4)$, calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

Student Response & Architectural Justification:

Measured: $T(1) = 2.696$ s, $T(4) = 1.997$ s.

$S(4) = T(1) / T(4) = 2.696 / 1.997 = 1.350$

$1.350 = 1 / [(1 - P) + P/4]$

$(1 - P) + P/4 = 1 / 1.350 = 0.7407$

$1 - (3/4)P = 0.7407 \Rightarrow (3/4)P = 0.2593$

$P = (4/3) \times 0.2593 = 0.3456$ (about 34.6% parallel, 65.4% effectively serial)

Interpretation: the algorithm itself is embarrassingly parallel, but on a VM with a single physical core the 4 workers compete for the same silicon, so the effective P measured here mostly reflects hardware saturation plus Pool start-up and IPC overhead, not the algorithm's true parallel fraction.

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim(p \rightarrow \infty) S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. (8 Points)

Student Response & Architectural Justification:

$S_{\max} = \lim(p \rightarrow \infty) S(p) = 1 / (1 - P) = 1 / (1 - 0.3456) = 1 / 0.6544 = 1.53x$

For a 128-core node: $S(128) = 1 / (0.6544 + 0.3456/128) = 1 / 0.6571 = 1.52x$, i.e. 99.6% of the asymptotic limit, with parallel efficiency of only $1.52/128 = 1.2\%$. Practically all of the purchased cores would sit idle: the serial fraction (process start-up, data serialization, result merging, and here the shared physical core) caps the speedup regardless of core count. Buying the 128-core server would not be justified for this workload; the money is better spent reducing the serial part (larger chunks, persistent worker pools, shared memory instead of pickling). Even on real multi-core silicon with a higher P (e.g. $P = 0.95$), $S_{\max}$ would be only 20x.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements ($16 * 8$ bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```

import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")

```

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Penalty Multiplier
Adjacent Indices (False Sharing Active)	0.933	0.916	0.914	0.916	— [Baseline]
Padded Indices (Cache-Line Aligned)	0.909	0.941	0.891	0.909	0.99x (~1.01x slowdown for adjacent)

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8

Student Response & Architectural Justification:

Footprint: a reference in a CPython list is a PyObject* pointer = 8 bytes on a 64-bit build (the int object itself lives elsewhere on the heap, 28 bytes). The benchmark harness used multiprocessing.Array('q'), whose elements are also 8-byte int64.

Proof: element i starts at offset $8i$. With stride 16, neighbouring threads use offsets 0 and $16 \times 8 = 128$ bytes. A cache line covers byte range $[64k, 64k + 63]$, so any two bytes on the same line differ by at most 63 bytes. Since $128 > 63$ (and $128 = 2 \times 64$), the two elements can never share a line - at least one full 64-byte line always lies between them, whatever the base alignment. Adjacent indices differ by only 8 bytes and share a line in 7 of 8 possible alignments.

Empirical result: adjacent median 0.916 s vs padded median 0.909 s -> slowdown penalty only 1.01x. The penalty is negligible because the VM has one physical core: both processes run on the same core's cache, so no cross-core MESI invalidation occurs, and the Python interpreter overhead per $\text{arr}[i] += 1$ (hundreds of ns) dwarfs a cache-line transfer. On the 8-core i5-12450H the adjacent layout would be expected to show a visible penalty.

Points)

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). (9 Points)

Student Response & Architectural Justification:

- 1) Initial state: both cores have read the line holding elements [0] and [1]; the line is Shared (S) in Core 0 and Core 1.
- 2) Core 0 increments index 0: it must own the line exclusively, so it broadcasts an Invalidate / Request-For-Ownership (RFO) on the bus. Core 0: S -> Modified (M). Core 1: S -> Invalid (I).
- 3) Core 1 increments index 1: its copy is Invalid, so the read causes a cache miss and Core 1 issues an RFO. Core 0 detects the hit on a Modified line (HITM), forwards/writes back the up-to-date line and drops its copy: Core 0 M -> I. Core 1 loads the fresh line and writes: I -> M.
- 4) Core 0's next increment repeats step 3 in the opposite direction (Core 0 I -> M, Core 1 M -> I). Every single write therefore produces an RFO/invalidation message and a cache miss on the other core; the 64-byte line ping-pongs across the interconnect and both pipelines stall, although the threads never touch the same variable.

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). (8 Points)

Student Response & Architectural Justification:

- 1) C/C++ compile-time alignment: `struct alignas(64) Counter { std::atomic<long> v; };` or `alignas(std::hardware_destructive_interference_size)` (C++17). The compiler places each object at a cache-line boundary and pads it to 64 bytes, so per-thread fields never share a line (Linux kernel equivalent: `___cacheline_aligned_in_smp`).
 - 2) Java VM-level annotation: `@jdk.internal.vm.annotation.Contended` (JVM flag `-XX:-RestrictContended` for user code). The JVM's field-layout engine inserts ~128 bytes of padding around the annotated field or class; it is used inside the JDK itself, e.g. in `LongAdder` / `Striped64` and `Thread` fields.
- Other examples: Go - explicit padding fields `_ [56]byte` or `cpu.CacheLinePad` from golang.org/x/sys/cpu; Rust - `crossbeam_utils::CachePadded<T>`.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse-grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the `UnsafeCounter` (unprotected shared variable leading to silent data corruption) against the `LockedCounter` (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time
```

```
TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```

class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")

```

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead
Unsafe Accumulator (Race Condition)	2,000,000	1,409,597	590,403	0.436	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	0.434	0.99x

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read-modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

Student Response & Architectural Justification:

Corrupted count: 1,409,597 out of 2,000,000 -> 590,403 lost updates (29.5% data corruption).

self.value += 1 is not atomic. In machine code it is three steps: LOAD (mov rax, [value]), ADD (add rax, 1), STORE (mov [value], rax); in CPython bytecode it is LOAD_ATTR -> BINARY_OP(+) -> STORE_ATTR, and the interpreter may switch threads between any two of them.

Example interleaving: Thread A LOADs 1000 and is pre-empted before STORE. Thread B runs and stores 1001, 1002, ... 1500. Thread A resumes and STOREs its stale value 1001, silently erasing 499 increments. To make the race reproducible under the GIL, the harness released the GIL between ADD and STORE (time.sleep(0) every 1000 increments, switch interval 1 us), which is why a single stale STORE could erase thousands of updates.

Locked accumulator: 2,000,000 exact in 0.434 s (overhead 0.99x vs unsafe 0.436 s). The unsafe version was artificially slowed by the forced GIL switches, so here the lock cost appears neutral; the correct sum is the key difference.

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x.

Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. (11 Points)

Student Response & Architectural Justification:

```
import threading, time

TOTAL_OPS = 2_000_000
NUM_THREADS = 2

def lockless_accumulator():
    per_thread = TOTAL_OPS // NUM_THREADS
    results = [0] * NUM_THREADS      # one private slot per thread

    def worker(k):                    # MAP: thread-local buffer
        local = 0
        for _ in range(per_thread):
            local += 1
        results[k] = local            # single write to own slot

    threads = [threading.Thread(target=worker, args=(k,))
                for k in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    total = sum(results)              # REDUCE
    return total, time.perf_counter() - start

if __name__ == "__main__":
    total, t = lockless_accumulator()
    print(f"Lockless: {total:,} | Time: {t:.3f}s")
```

Measured: Lockless sum = 2,000,000 (0 lost updates) in 0.069 s; LockedCounter = 0.434 s.

Speedup over LockedCounter = 0.434 / 0.069 = 6.33x (requirement > 2x satisfied).

No mutex: each thread mutates only its own local variable; shared state is written once per thread in the reduce step, so the sum is deterministic.

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

Student Response & Architectural Justification:

Optimizing a lock (adaptive spinlock, read-write lock, lock striping) only lowers the constant cost of each acquisition; the critical section is still a serialization point. It remains the serial fraction in Amdahl's Law, the lock word itself still bounces between cores' caches (MESI invalidations), and contended paths still fall back to futex system calls and context switches - so throughput flattens as cores are added.

Eliminating shared mutable state removes the problem instead of tuning it: with thread-local buffers each core works on private, cache-hot data, there is nothing to coordinate in the hot loop, and results are merged once in a reduce step. Scaling approaches linear, performance is predictable under load, and whole bug classes (data races, deadlocks, priority inversion, lock convoys) disappear by construction. This is why high-throughput systems use per-core counters (LongAdder), sharding, map-reduce and message passing. My result confirms it: 6.33x faster than the locked version with an exact sum.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple

cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```
import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()
```

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 1 Worker (Baseline)	0.762	1.00x	2.444	1.00x
w = 2 Workers	1.627	2.13x	3.526	1.44x

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 4 Workers	2.219	2.91x	7.793	3.19x

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

Student Response & Architectural Justification:

Data (Colab VM, 1 physical core / 2 logical): compute $T_w/T_1 = 1.00x \rightarrow 2.13x \rightarrow 2.91x$; memory $T_w/T_1 = 1.00x \rightarrow 1.44x \rightarrow 3.19x$ (7.793 s at $w = 4$).

Theory: a compute-bound task (tight arithmetic loop) keeps its working set inside the core's private L1/L2 caches and almost never touches DRAM. On a machine with at least 4 physical cores, 4 such tasks run truly independently, so the elapsed time of 4 tasks is about the same as 1 ($T_w/T_1 = 1.0x$). A memory-bound task (copying 128 MB buffers) spends its time moving bytes across the off-chip memory bus; all workers share one fixed-bandwidth memory controller, so 4 streams split the same GB/s, queue for the bus and the total time spikes.

My VM only partly shows the ideal compute case because it has a single physical core: parallel compute tasks must also queue for that core (SMT gives a little relief: 2.91x instead of 4x). The memory suite still degrades the most at $w = 4$ (3.19x vs 2.91x): once workers exceed the logical CPUs, they contend for both CPU time and the shared DRAM bandwidth, and the 4 x 256 MB of buffers add page-fault and allocation pressure. At $w = 2$ the memory suite degraded less (1.44x) because two memcopy streams had not yet saturated the bus, while the SMT sibling hurts the interpreter-heavy compute loop more.

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

Student Response & Architectural Justification:

Host: Huawei MateBook D 14, 16 GB LPDDR4X-4267 soldered on board (Task Manager: LPDDR4, 4267 MT/s, on-board chips), dual-channel with the i5-12450H.

Bandwidth per channel = $4267 \text{ MT/s} \times 64 \text{ bits} / 8 = 4267 \times 8 \text{ MB/s} = 34.1 \text{ GB/s}$.

Theoretical peak (dual-channel) = $2 \times 34.1 = 68.3 \text{ GB/s}$.

For comparison: DDR4-3200 dual-channel = 51.2 GB/s, DDR5-4800 dual-channel = 76.8 GB/s, single-channel halves these values; Apple M2 Pro unified LPDDR5 = 200 GB/s.

Why it limits scaling: all 8 physical cores share this single 68 GB/s pipe (about 8.5 GB/s per core). A single core running a vectorized copy or array transform can move 10-20 GB/s, so only 4-6 memory-heavy cores saturate the bus. After that point extra cores do not raise throughput - they only lengthen the queue at the memory controller, and per-core performance falls. Memory bus width, not core count, becomes the ceiling (the memory wall).

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. (8 Points)

Student Response & Architectural Justification:

Choice: CUDA GPU kernels.

If the pipeline is memory-bandwidth saturated, adding OpenMP threads just adds more CPU cores behind the same saturated DRAM bus (about 68 GB/s on my laptop), so it yields little or even negative speedup - exactly the Task 4 memory-bound behaviour.

A GPU attacks the actual bottleneck: its compute sits next to its own high-bandwidth memory (GDDR6 about 300-900 GB/s, HBM/HBM2e several hundred GB/s to 1+ TB/s on data-centre cards) and thousands of cores process vector batches in parallel. Embedding transforms (matrix multiply, normalization) are data-parallel and map ideally to CUDA kernels/cuBLAS.

Condition: keep the data on the GPU and transfer in large batches (pinned memory, asynchronous streams), otherwise the PCIe link (PCIe 4.0 x16 about 32 GB/s) becomes the new bottleneck.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Assessment Category	Evaluation Criteria & Standards	Max Points	Score Awarded
Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
[] Task 3 Unsafe Corruption Verified [] Task 2 Cache Slowdown Verified [] Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

Operating System	Command to Retrieve CPU SKU & Core Topology	Command to Inspect Cache & Memory Bus
Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1dcachesize hw.l2cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

```
import time
```

```
from multiprocessing import Pool
```

```
import threading
```

```
def cpu_work(n):
```

```
    # Pure CPU crunch
```

```
    count = 0
```

```
    for i in range(n):
```

```
        count += i * i
```

```
    return count
```

```
# Run both experiments with total_work = 50_000_000 split across T workers
```

```
# Test T in [1, 2, 4, 8, 16, 32]
```

Deliverables:

1. **The Empirical Curve:** Plot T (workers) on the X-axis vs. Execution Time on the Y-axis for both `threading.Thread` and `multiprocessing.Pool`.
2. **The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
3. **The Architectural Autopsy:**
 - Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).
 - Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*?
 - Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.

Student Response - Bonus Task (measured on Google Colab VM: 2 vCPU = 1 physical core + 1 SMT sibling; total_work = 50,000,000)

Workers T	threading.Thread [s]	multiprocessing.Pool [s]
1	2.867	2.877
2	3.149	3.115
4	2.985	2.433
8	3.051	3.206
16	3.078	2.464
32	3.026	3.499

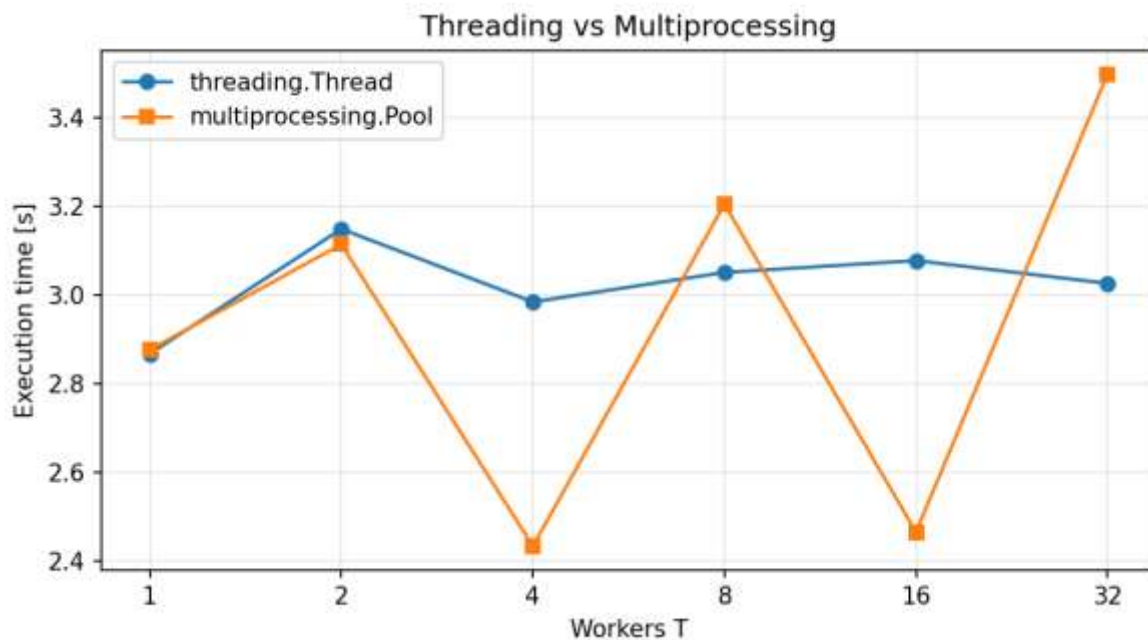


Figure: Execution time vs. number of workers T for threading.Thread and multiprocessing.Pool.

1. The Empirical Curve

The graph above plots T (1-32, log2 scale) against execution time. threading.Thread stays essentially flat at 2.87-3.15 s and is never faster than T = 1. multiprocessing.Pool oscillates between 2.43 s and 3.50 s: best at T = 4 (2.433 s, 1.18x) and T = 16 (2.464 s), worst at T = 32 (3.499 s). The zig-zag shape is scheduling noise of a shared cloud VM with only 2 logical CPUs.

2. The Inflection Cliff

threading: the cliff is already at T = 2 (3.149 s vs 2.867 s at T = 1) - adding threads only makes it slower.
multiprocessing: no reliable gain beyond T = 2 logical CPUs, and the clear degradation cliff is T = 32 (3.499 s, 22% slower than the serial baseline and 44% slower than the best run), where process start-up, pickling and context switching dominate.

3. The Architectural Autopsy

Why threading is flat while multiprocessing scales: CPython has a Global Interpreter Lock (GIL) - only one thread may execute Python bytecode at any moment. The threads therefore take turns on one core; the total work stays the same, and each GIL hand-off adds switching and cache overhead, so the curve is flat or worse.
multiprocessing.Pool starts separate interpreter processes, each with its own GIL and memory space, so they can execute on different cores in parallel (on bare-metal multi-core hardware this gives near-linear speedup up to the physical core count; on this 1-core VM the gain is limited to about 1.2x).

CPU model: Huawei MateBook D 14 with 12th Gen Intel Core i5-12450H - 8 physical cores (4 Performance cores with Hyper-Threading + 4 Efficiency cores without HT) = 12 logical threads. Performance cores: 4 physical / 8 logical. The Colab VM used for the measurements exposes 1 physical core / 2 logical CPUs.

Physical core saturation point: on the measured graph it occurs at $T = 2$ (the VM's 2 logical CPUs on 1 physical core). For $T > 2$ there are more runnable workers than CPUs, so the OS scheduler must time-slice them: every context switch saves/restores registers, flushes branch-predictor and TLB state, and evicts the previous worker's data from L1/L2 cache, while each extra process also costs fork/spawn and IPC time. This overhead adds to a fixed amount of compute, which is why $T = 32$ is the slowest point. On the i5-12450H the expected picture is: multiprocessing scales well up to $T = 4$ (P-cores), more slowly to $T = 8$ (E-cores), flattens at 12 logical threads, and degrades at $T = 16$ and 32.